파이썬 완전 백과사전 2권 · 머신러닝 기초 교안

2일차

모델을 고르고 제대로 재기

6~10차시 · 책 2장 2.7절 ~ 3장

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/02-data-preparation, 03-classic-ml

오늘의 시간표

차시	주제	책 범위
6	Pipeline, 불균형, 시간 데이터와 이탈 실습	2.7 ~ 2.10
7	선형 회귀와 로지스틱 회귀	3장 들어가며 ~ 3.3
8	결정 트리, 랜덤 포레스트·부스팅, k-means	3.4 ~ 3.6
9	평가 지표와 교차 검증	3.7 ~ 3.9
10	하이퍼파라미터 탐색, 저장, 종합 실습	3.10 ~ 3.13

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2 폴더에서 uv sync · 10차시 캘리포니아 집값 데이터만 처음 실행 때 내려받는다

6차시

06

Pipeline, 불균형, 시간 데이터와 이탈 실습

책 2.7절 ~ 2.10절

학습 목표

- 열마다 다른 손질과 모델을 Pipeline 하나로 묶는다
- 드문 쪽이 중요한 데이터를 stratify와 클래스 가중치로 다룬다
- 시간 순서가 있는 데이터를 섞지 않고 시점으로 자른다

2.7 열마다 다른 손질: ColumnTransformer

```
숫자_처리 = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
범주_처리 = make_pipeline(SimpleImputer(strategy="constant", fill_value="모름"),
                          OneHotEncoder(handle_unknown="ignore", sparse_output=False))
전처리 = ColumnTransformer([("숫자", 숫자_처리, 숫자_열), ("범주", 범주_처리, 범주_열),
                            ("순서", 순서_처리, 순서_열)])
전처리.set_output(transform="pandas")
결과 = 전처리.fit_transform(매물)
```

	숫자__면적	숫자__층	범주__구_강서	범주__구_마포	범주__구_모름	범주__구_송파	순서__상태
0	0.25	-0.05	0.0	1.0	0.0	0.0	1.0
1	-0.96	-1.36	1.0	0.0	0.0	0.0	2.0
2	0.25	1.84	0.0	1.0	0.0	0.0	0.0
3	1.67	-0.78	0.0	0.0	1.0	0.0	2.0
4	0.25	-0.05	0.0	0.0	0.0	1.0	1.0
5	-1.45	0.39	1.0	0.0	0.0	0.0	1.0

2.7 손질부터 요리까지 한 번에: Pipeline

```
전처리 = ColumnTransformer([
    ("숫자", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
     ["나이", "주간방문"]),
    ("범주", OneHotEncoder(handle_unknown="ignore"), ["지점"]),
])
모델 = Pipeline([("전처리", 전처리), ("분류", LogisticRegression())])
모델.fit(X_학습, y_학습)              # 전처리도 모델도 학습 데이터로만 배운다
print(f"시험 정확도: {모델.score(X_시험, y_시험):.1%}")
새_회원 = pd.DataFrame({"나이": [np.nan, 58.0], "주간방문": [4.0, 0.0], "지점": ["판교", "강남"]})
print("새 회원 재등록 확률:", 모델.predict_proba(새_회원)[:, 1].round(2))
```

```
시험 정확도: 77.0%
새 회원 재등록 확률: [0.84 0.4 ]
배운 중앙값: [44.  2.]
```

- **fit**: 학습 데이터로 손질법을 배우고, 바꾸고, 모델이 배운다
- **score, predict**: 시험·새 데이터에는 transform만
- 나이가 빈 "판교" 회원도 원본 모양 그대로 넣었다

2.8 불균형 데이터: 비율을 지키며 나누기

```
y = np.array([1] * 10 + [0] * 90)      # 100명 중 10명만 양성 (10%)
X = np.arange(100).reshape(-1, 1)

for 씨앗 in range(5):
    _, _, _, 그냥 = train_test_split(X, y, test_size=0.2, random_state=씨앗)
    _, _, _, 층화 = train_test_split(X, y, test_size=0.2, random_state=씨앗, stratify=y)
    print(f"씨앗 {씨앗}: 시험 20명 중 양성  그냥 {그냥.sum()}명, stratify {층화.sum()}명")
```

```
씨앗 0: 시험 20명 중 양성  그냥 3명, stratify 2명
씨앗 1: 시험 20명 중 양성  그냥 0명, stratify 2명
씨앗 2: 시험 20명 중 양성  그냥 3명, stratify 2명
씨앗 3: 시험 20명 중 양성  그냥 2명, stratify 2명
씨앗 4: 시험 20명 중 양성  그냥 1명, stratify 2명
```

그냥 나누면 양성이 **0~3명**으로 들쭉날쭉. 0명이면 "양성을 잘 찾는가"를 잴 수가 없다. 분류라면 언제나 **stratify=y**

2.8 클래스 가중치: 드문 쪽에 배점을 높게

```
X, y = make_classification(n_samples=4000, n_features=8, n_informative=4,
                           weights=[0.95], flip_y=0.01, class_sep=1.5, random_state=0)
for 가중치 in [None, "balanced"]:
    예측 = LogisticRegression(class_weight=가중치).fit(X_학습, y_학습).predict(X_시험)
    찾음 = ((예측 == 1) & (y_시험 == 1)).sum()
    헛경보 = ((예측 == 1) & (y_시험 == 0)).sum()
```

```
시험 데이터 1000건 중 양성 55건
class_weight=None      정확도 96.0%  양성 15건 찾음, 헛경보 0건
class_weight=balanced  정확도 76.1%  양성 43건 찾음, 헛경보 227건
```

암 검진: 헛경보가 좀 많아도 환자를 놓치지 않는 쪽

스팸 필터: 중요한 메일을 스팸함에 넣는 헛경보가 더 치명적

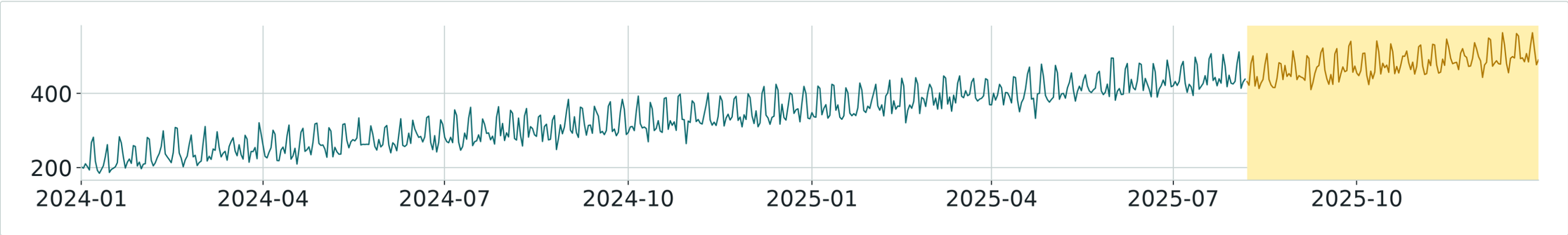
6차시 · 2.8 클래스 가중치 맛보기 · 예제 02-data-preparation/b22_class_weight.py (핵심 부분)

2.9 시간 데이터: 내일 신문으로 어제를 맞히기

```
섞은_학습, 섞은_시험 = train_test_split(일차, test_size=0.2, random_state=0)
경계 = int(len(일차) * 0.8)                              # 앞 80%로 배우고 뒤 20%로 시험
```

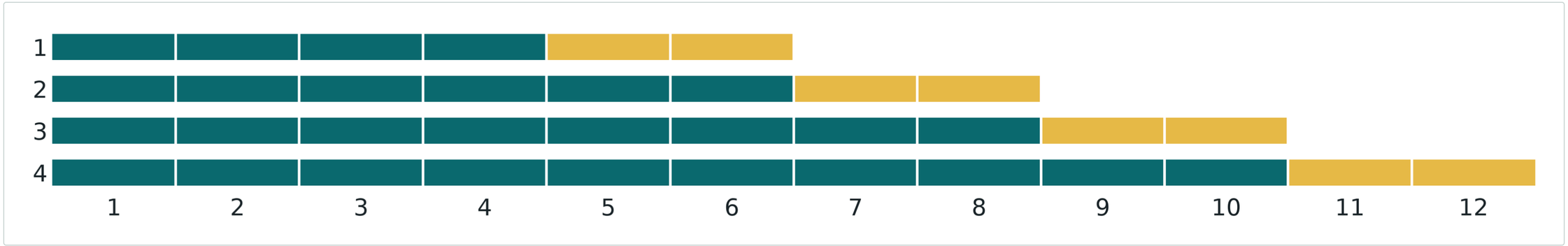
```
마구 섞어 나누기: 평균  14.0개 틀림
시간 순서로 자르기: 평균  34.8개 틀림
시험 구간: 2025-08-07 ~ 2025-12-31
```

섞으면 시험 날짜의 **바로 앞뒤 날**이 학습에 들어간다. 정직한 점수는 34.8.
아래 그림의 노란 구간이 시간 순서로 자른 시험 구간



2.9 시간을 따라 여러 번 나누기: TimeSeriesSplit

```
달 = pd.period_range("2025-01", "2025-12", freq="M")
for 회차, (학습, 시험) in enumerate(TimeSeriesSplit(n_splits=4).split(달), start=1):
    print(f"{회차}회차  학습 {달[학습[0]]}~{달[학습[-1]]}  →  시험 {달[시험[0]]}~{달[시험[-1]]}")
```



가로는 2025년 1~12월, 세로는 회차. **청록 = 학습**, **노랑 = 시험**. 학습 구간을 늘려 가며 언제나 그 바로 뒤로 시험

6차시 · 2.9 시간 순서가 있는 데이터 · 예제 02-data-preparation/b24_timeseries_split.py (실행 출력을 그림으로)

2.10 실습: 통신사 고객 이탈 데이터 손질하기

```
학습: 1800명, 이탈 10.7%
검증: 600명, 이탈 10.7%
테스트: 600명, 이탈 10.7%
[검증] class_weight=None      정확도 89.3%, 이탈자 64명 중 7명 찾음
[검증] class_weight=balanced 정확도 69.8%, 이탈자 64명 중 41명 찾음
[테스트] 정확도 72.8%, 이탈자 64명 중 47명 찾음
전처리 후 특성: 15 개
새 고객 이탈 확률: [0.97 0.06]
```

나누기 전에 고쳐도 된다

중복 25행, 지역 18가지 표기, 999살 39명. 행 하나만 보고 고친다

나눈 뒤 학습 데이터로

중앙값 채우기, 표준화, 원-핫 범주 목록. Pipeline 안에서

만들기(b25) → 살펴보고 행 단위로 고치기(b26) → 나누고, 묶고, 고르고, 채점하기(b27)

용어: 2장의 **테스트** 데이터 = 3장(7차시부터)의 **시험 세트**. 둘 다 고르기에 쓰지 않고 마지막에 한 번만 채점하는 몫

직접 실행 10분

1. 02-data-preparation 폴더에서 b25 → b26 → b27을 차례로 실행한다. b25가 data/통신사_고객.csv를 만들어야 뒤 둘이 돈다
2. b22_class_weight.py의 후보 목록을 [None, {0: 1, 1: 5}, "balanced"]로 바꾸고 다시 실행한다. 가운데 줄은 어디쯤에 놓이나?
3. b27 출력에서 검증(41명)과 테스트(47명)를 견주어, 이 점수를 믿어도 되는지 말해 본다

```
cd examples/vol2/02-data-preparation && uv run python b25_practice_make.py
```

2년치 하루 판매량으로 다음 달을 예측하려 합니다. 어떻게 나눌까요?

Q

가 train_test_split 기본값으로 무작위로 나눈다

나 stratify를 주고 무작위로 나눈다

다 날짜순으로 정렬해 앞은 학습, 뒤는 시험으로 자른다

라 요일별로 고르게 섞은 뒤 나눈다

6차시 정리

ColumnTransformer로 열마다 다른 손질, Pipeline으로 손질과 모델을 하나로. 누수 방지가 코드 구조로 보장된다

불균형: 정확도만 보면 "늘 정상"도 높은 점수. stratify로 나누고 class_weight로 드문 쪽 배점 올리기

시간 데이터는 섞지 말고 시점으로 자르기. 고르기는 검증으로, 테스트는 마지막에 한 번

다음 차시: 3장으로 · 숫자를 예측하는 직선과 확률로 답하는 분류

7차시

07

선형 회귀와 로지스틱 회귀

책 3장 들어가며 ~ 3.3절

학습 목표

- fit, predict, score 약속으로 모델을 바꿔 끼운다
- 선형 회귀의 계수를 읽고, 릿지·라소가 왜 계수에 고삐를 매는지 말한다
- 로지스틱 회귀가 점수를 확률로 바꾸는 과정을 설명한다

3.1 모든 모델이 같은 손잡이: fit, predict, score

```
모델들 = {
    "최근접 이웃": make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "로지스틱 회귀": make_pipeline(StandardScaler(), LogisticRegression()),
    "결정 트리": DecisionTreeClassifier(random_state=42),
}
for 이름, 모델 in 모델들.items():
    모델.fit(X_학습, y_학습)                # 배우기: 모델마다 똑같은 한 줄
    정확도 = 모델.score(X_시험, y_시험)     # 처음 보는 문제로 채점하기
    print(f"{이름}: 시험 정확도 {정확도:.3f}")
```

```
최근접 이웃: 시험 정확도 0.979
로지스틱 회귀: 시험 정확도 0.986
결정 트리: 시험 정확도 0.923
```

- **모델종류(설정=값)**: 사람이 정하는 하이퍼파라미터
- **fit · predict · predict_proba · score**: 배우고, 답하고, 확률 내고, 채점
- **이름_**: 끝에 밑줄이 붙은 속성은 fit으로 배운 결과

3.2 선형 회귀: 집값의 숨은 공식 찾기

```
rng = np.random.default_rng(0)
넓이 = rng.uniform(10, 40, size=50)
가격 = 1.0 + 0.15 * 넓이 + rng.normal(0, 0.3, size=50)   # 숨은 규칙 + 잡음

모델 = LinearRegression()
모델.fit(넓이.reshape(-1, 1), 가격)     # X는 언제나 (샘플 수, 특성 수) 2차원
print(f"기울기(평당 가격): {모델.coef_[0]:.3f}억 원")
print(f"절편(기본 가격):   {모델.intercept_:.3f}억 원")
print("30평, 35평 예상 가격:", 모델.predict([[30], [35]]).round(2))
```

```
기울기(평당 가격): 0.155억 원
절편(기본 가격):   0.875억 원
30평, 35평 예상 가격: [5.53 6.31]
```

숨긴 규칙 **1.0 + 0.15 × 넓이**, 점 50개로 찾은 규칙 **0.875 + 0.155 × 넓이**

3.2 특성이 여러 개일 때: 계수 읽기

```
모델 = make_pipeline(StandardScaler(), LinearRegression())
모델.fit(X_학습, y_학습)

선형 = 모델[-1]                              # 파이프라인의 마지막 단계 꺼내기
계수 = sorted(zip(당뇨.feature_names, 선형.coef_), key=lambda 쌍: -abs(쌍[1]))
for 이름, 값 in 계수:
    print(f"{이름:>4}: {값:+7.1f}")
```

```
  s1:   -44.1
  s5:   +33.4
 bmi:   +25.1
  s2:   +24.5
  bp:   +18.2
  s4:   +13.0
 sex:   -11.5
  s3:    +5.5
 age:    +2.2
  s6:    +1.2
절편: 154.3
시험 세트 R²: 0.485
```

bmi +25.1: 상식과 맞다. 그런데 비슷하게 움직이는 **s1 -44.1, s2 +24.5**는 서로 밀고 당긴다. 다중공선성

3.2 계수에 고삐 매기: 릿지와 라소

```
for 이름, 회귀 in [("선형", LinearRegression()), ("릿지", Ridge(alpha=10.0)),
                   ("라소", Lasso(alpha=1.0))]:
    모델 = make_pipeline(StandardScaler(), 회귀).fit(X_학습, y_학습)
    계수 = 모델[-1].coef_.round(1) + 0.0          # + 0.0은 -0.0을 0.0으로 정리
```

```
          age   sex   bmi    bp    s1    s2    s3    s4    s5    s6   시험 R²
선형      2.2 -11.5  25.1  18.2 -44.1  24.5   5.5  13.0  33.4   1.2   0.485
릿지      2.4 -11.0  25.1  17.6 -13.1   0.5  -7.8   8.5  21.2   1.9   0.488
라소      1.2  -9.3  25.7  17.0  -8.4   0.0 -10.8   3.3  21.1   1.1   0.496
```

릿지: 계수 제곱에 벌점. s1·s2가 -13.1과 0.5로 얌전해졌다

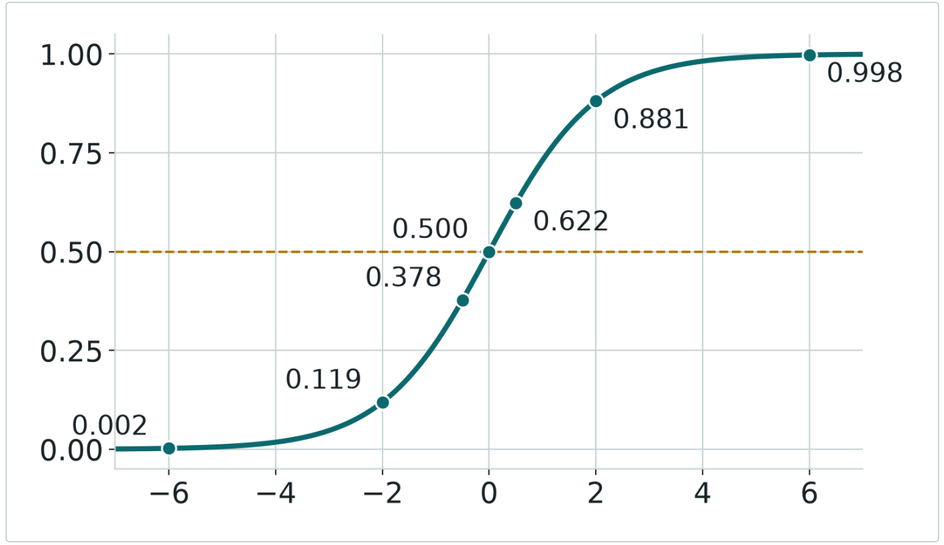
라소: 절댓값에 벌점. s2를 아예 0으로 지웠다

3.3 점수를 확률로 바꾸는 S자 곡선

```
def 시그모이드(z: np.ndarray) -> np.ndarray:
    """어떤 실수든 0과 1 사이 값으로 눌러 담는다."""
    return 1 / (1 + np.exp(-z))

점수 = np.array([-6.0, -2.0, -0.5, 0.0, 0.5, 2.0, 6.0])
for z, p in zip(점수, 시그모이드(점수)):
    print(f"점수 {z:+5.1f} → 확률 {p:.3f}")
```

로지스틱 회귀 = 선형 회귀로 점수 → 시그모이드로 확률 → 0.5를 넘으면 1



가로: 점수 z · 세로: 확률 · 노란 점선: 문턱 0.5 · 점 옆 숫자는 실행 출력값

7차시 · 3.3 로지스틱 회귀: 확률로 답하는 분류 · 예제 03-classic-ml/b06_sigmoid.py (출력은 그림으로)

3.3 유방암 진단에 써 보기

```
모델 = make_pipeline(StandardScaler(), LogisticRegression())
모델.fit(X_학습, y_학습)

확률 = 모델.predict_proba(X_시험[:5])     # 열 0: 악성 아님, 열 1: 악성
print("예측 확률:\n", 확률.round(3))
print("예측 레이블:", 모델.predict(X_시험[:5]))
print("실제 레이블:", y_시험[:5])

계수 = 모델[-1].coef_[0]                    # 특성 30개에 하나씩
for i in 계수.argsort()[::-1][:3]:          # 큰 것부터 세 개
    print(f"{암.feature_names[i]:>15}: {계수[i]:+.2f}")
```

```
예측 확률:
 [[0.001 0.999]
 [0.996 0.004]
 [0.    1.   ]
 [1.    0.   ]
 [0.965 0.035]]
예측 레이블: [1 0 1 0 0]
실제 레이블: [1 0 1 0 0]
  worst texture: +1.37
   radius error: +1.27
 worst symmetry: +1.04
```

로지스틱 회귀는 **층이 하나뿐인 신경망**: 입력 × 가중치 + 편향, 그리고 시그모이드

직접 실행 10분

1. 03-classic-ml 폴더에서 b01_same_api.py를 실행하고, 모델 셋이 같은 두 줄(fit, score)로 도는지 확인한다
2. b05_ridge_lasso.py에서 Lasso(alpha=1.0)을 Lasso(alpha=5.0)으로 바꿔 실행한다. 계수가 0이 된 특성은 몇 개이고, 끝까지 남은 특성은 무엇인가?
3. b07_logistic.py를 실행해 다섯 환자의 악성 확률과 예측 레이블이 0.5 문턱으로 어떻게 이어지는지 짚는다

```
cd examples/vol2/03-classic-ml && uv run python b05_ridge_lasso.py
```

로지스틱 회귀에 대한 설명으로 맞는 것은?

가 이름처럼 숫자를 예측하는 회귀 모델이다

나 점수를 시그모이드로 0~1 확률로 바꾸고, 0.5를 넘으면 1이라고 답한다

다 predict_proba의 두 칸을 더하면 환자마다 다르다

라 계수가 음수인 특성은 클수록 악성 쪽으로 민다

7차시 정리

약속: fit으로 배우고 predict·predict_proba로 답하고 score로 채점. 끝에 밑줄이 붙은 속성은 배운 결과

선형 회귀는 계수와 절편을 배우고, **릿지·라소**는 계수에 벌점을 줘 고삐를 맨다

로지스틱 회귀는 점수를 시그모이드로 확률로 바꾸는 분류 모델. 층이 하나뿐인 신경망

다음 차시: 스무고개, 다수결, 오답 노트, 깃발 세 개

8차시

08

결정 트리, 랜덤 포레스트·부스팅, k-means

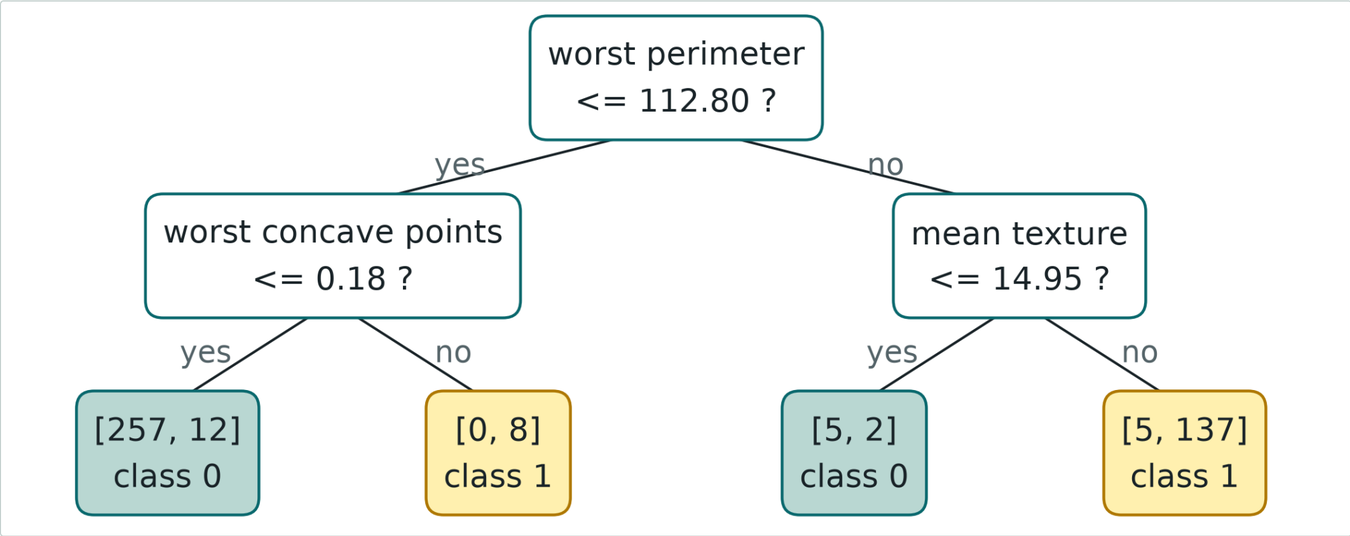
책 3.4절 ~ 3.6절

학습 목표

- 결정 트리가 고른 질문을 읽고, 깊은 나무가 외우는 모습을 숫자로 본다
- 랜덤 포레스트(다수결)와 부스팅(오답 노트)의 차이를 말한다
- k-means로 정답 없이 묶고, 무리 수와 눈금 맞추기를 따진다

3.4 결정 트리: 스무고개로 답 찾기

```
나무 = DecisionTreeClassifier(max_depth=2, random_state=42)
나무.fit(X_학습, y_학습)
print(export_text(나무, feature_names=암.feature_names.tolist(), show_weights=True))
print(f"질문 두 번만으로 시험 정확도 {나무.score(X_시험, y_시험):.3f}")
```



질문 두 번만으로 시험 정확도 0.895. 잎의 [a, b]는 도착한 학습 환자 수(악성 아님, 악성)

"이 값보다 큰가 작은가"만 물으니 **표준화가 필요 없다**

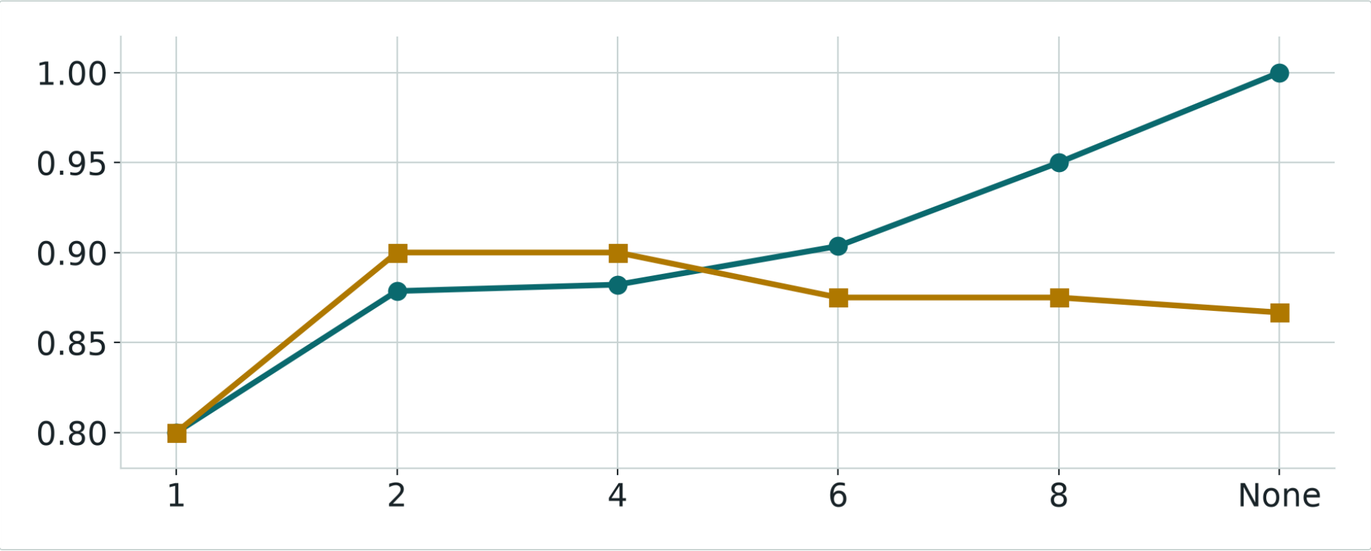
8차시 · 3.4 결정 트리 · 예제 03-classic-ml/b08_tree_rules.py (핵심 부분, export_text 출력을 그림으로)

3.4 너무 깊은 나무는 외워 버립니다

```
X, y = make_moons(n_samples=400, noise=0.35, random_state=0)
X_학습, X_시험, y_학습, y_시험 = train_test_split(X, y, test_size=0.3, random_state=0)
for 깊이 in [1, 2, 4, 6, 8, None]:
    나무 = DecisionTreeClassifier(max_depth=깊이, random_state=0).fit(X_학습, y_학습)
```

깊이	잎 수	학습 정확도	시험 정확도
1	2	0.800	0.800
2	4	0.879	0.900
4	12	0.882	0.900
6	23	0.904	0.875
8	32	0.950	0.875
None	51	1.000	0.867

청록 선 = 학습 정확도, 노랑 선 = 시험 정확도. 노란 줄 = 시험 정확도 최고(0.900)



8차시 · 3.4 너무 깊은 나무는 외워 버립니다 · 예제 03-classic-ml/b09_tree_depth.py (핵심 부분, 출력은 표와 그림으로)

3.5 랜덤 포레스트: 각자 풀고 다수결로

```
for 그루_수 in [1, 10, 50, 200]:
    숲 = RandomForestClassifier(n_estimators=그루_수,
         random_state=0).fit(X_학습, y_학습)
```

```
끝까지 자란 나무 한 그루: 시험 정확도 0.867
나무   1그루짜리 숲: 학습 0.939, 시험 0.833
나무  10그루짜리 숲: 학습 0.986, 시험 0.908
나무  50그루짜리 숲: 학습 1.000, 시험 0.900
나무 200그루짜리 숲: 학습 1.000, 시험 0.900
```

```
표 = sum(int(그루.predict(환자)[0])
          for 그루 in 숲.estimators_)
```

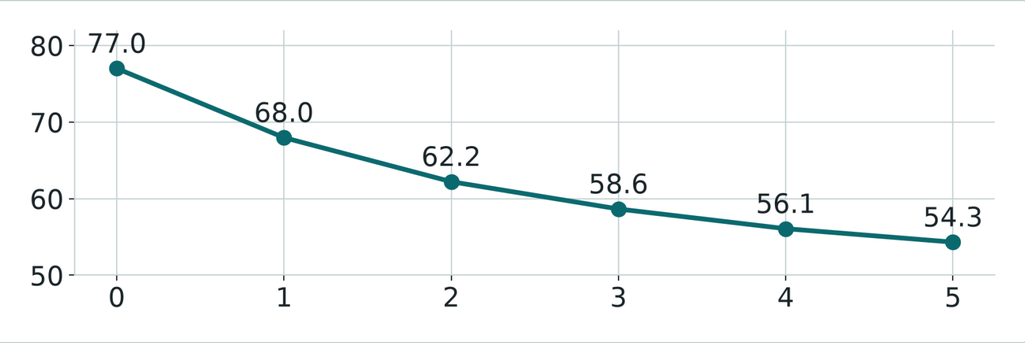
```
숲의 시험 정확도: 0.958
가장 애매한 환자: 악성 94표, 악성 아님 106표
     worst perimeter: 0.150
          worst area: 0.121
worst concave points: 0.103
 mean concave points: 0.101
```

나무 하나하나는 외우지만 **각자 다른 잡음**을 외웠기 때문에, 투표하면 실수가 묻힌다

8차시 · 3.5 랜덤 포레스트 · 예제 03-classic-ml/b10_forest_moons.py(왼쪽), 03-classic-ml/b10b_forest_votes.py(오른쪽) (코드는 줄바꿈만 바꾼 핵심 부분)

3.5 그래디언트 부스팅: 오답 노트를 이어서 고치기

```
예측 = np.full(len(y), y.mean())          # 0단계: 모두 평균으로 찍기
학습률 = 0.3
for 단계 in range(1, 6):
    잔차 = y - 예측                        # 지금까지 틀린 만큼 (오답 노트)
    작은_나무 = DecisionTreeRegressor(max_depth=2, random_state=0).fit(X, 잔차)
    예측 = 예측 + 학습률 * 작은_나무.predict(X)   # 오답을 조금만 고친다
    오차 = np.sqrt(np.mean((y - 예측) ** 2))
    print(f"{단계}단계 뒤 평균 오차(RMSE): {오차:.1f}")
```



가로: 단계(0 = 평균으로 찍기) · 세로: 평균 오차(RMSE)

숲은 각자 따로 키워 투표, **부스팅**은 앞 나무가 틀린 만큼을 다음 나무가 차례로 고친다

8차시 · 3.5 그래디언트 부스팅 · 예제 03-classic-ml/b11_boost_by_hand.py (핵심 부분, 출력은 그림으로)

3.5 실전 부스팅: HistGradientBoosting

```
부스팅 = HistGradientBoostingClassifier(random_state=42).fit(X_학습, y_학습)
print(f"그래디언트 부스팅 시험 정확도: {부스팅.score(X_시험, y_시험):.3f}")
print("실제로 만든 나무 수:", 부스팅.n_iter_)

# 값의 10%를 일부러 지워도(결측치) 그대로 학습한다
rng = np.random.default_rng(0)
X_구멍 = X_학습.copy()
X_구멍[rng.random(X_구멍.shape) < 0.1] = np.nan
부스팅.fit(X_구멍, y_학습)
```

```
그래디언트 부스팅 시험 정확도: 0.965
실제로 만든 나무 수: 100
빈칸 개수: 1311
빈칸 있는 데이터로 배운 뒤 시험 정확도: 0.972
```

설정 그대로 0.965. 빈칸(NaN)을 **채우지 않고** 그대로 받는다

8차시 · 3.5 그래디언트 부스팅 · 예제 03-classic-ml/b12_hgb.py (핵심 부분)

3.6 k-means: 깃발 세 개 주위로 모이기

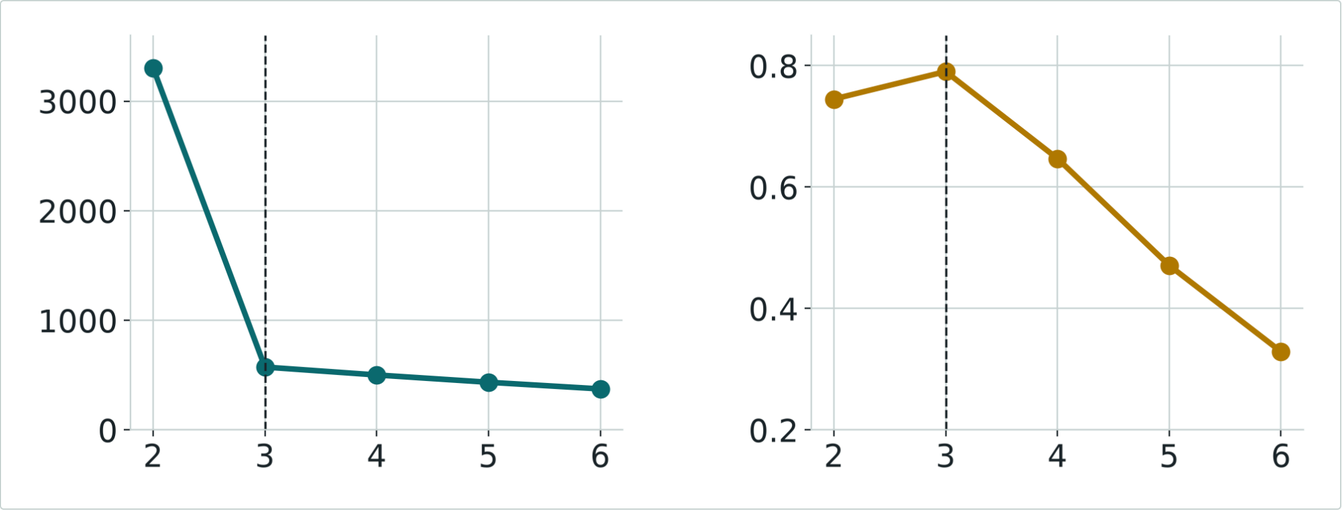
```
X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=7)
군집 = KMeans(n_clusters=3, n_init=10, random_state=0)
번호 = 군집.fit_predict(X)              # 배우고 곧바로 각 점의 무리 번호 받기
print("처음 10개 점의 무리 번호:", 번호[:10])
print("무리별 점 개수:", np.bincount(번호))
print("무리 중심:\n", 군집.cluster_centers_.round(2))
print("새 점 두 개의 무리:", 군집.predict([[9.0, 1.0], [-8.0, 5.0]]))
```

1. 학생들은 **가장 가까운 깃발**로 모인다
2. 선생님은 각 무리의 **한가운데**로 깃발을 옮긴다
3. 깃발이 더 움직이지 않을 때까지 1, 2를 반복

```
처음 10개 점의 무리 번호: [0 2 2 0 1 2 0 1 2 0]
무리별 점 개수: [100 100 100]
무리 중심:
 [[-8.55  5.57]
 [ 9.63  0.54]
 [-1.26  4.38]]
새 점 두 개의 무리: [1 0]
```

8차시 · 3.6 정답 없이 묶기: k-means 군집 · 예제 03-classic-ml/b13_kmeans.py (핵심 부분)

3.6 깃발은 몇 개를 꽂아야 할까?



왼쪽: 관성(청록) · 오른쪽: 실루엣 점수(노랑) · 가로: k · 점선: k=3

k	관성(inertia)	실루엣 점수
2	3302.8	0.745
3	573.4	0.790
4	501.4	0.647
5	434.8	0.471
6	374.2	0.329

관성은 k가 커지면 늘 준다. 줄어드는 폭이 꺾이는 곳(**엘보**)을 본다

실루엣 점수는 클수록 또렷. 둘 다 **k=3**을 가리킨다

8차시 · 3.6 깃발은 몇 개를 꽂아야 할까요? · 예제 03-classic-ml/b14_kmeans_k.py (출력은 표와 그림으로)

3.6 거리를 쓰는 모델은 눈금을 맞춰야 합니다

```
그대로 = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
맞춘_뒤 = make_pipeline(StandardScaler(),
                        KMeans(n_clusters=3, n_init=10, random_state=0)).fit_predict(X)
```

```
특성 범위 예: proline 278.0 ~ 1680.0 / nonflavanoid_phenols 0.13 ~ 0.66
스케일링 없이: 품종과 일치도 0.371
스케일링 후:   품종과 일치도 0.897
```

k-means는 거리가 전부. 범위가 큰 proline이 거리를 혼자 정하지 않게 **표준화**부터

결정 트리는 "이 값보다 큰가"만 물어 눈금에 영향을 받지 않는다

직접 실행 10분

1. b09_tree_depth.py를 실행하고, 학습 정확도와 시험 정확도가 벌어지기 시작하는 깊이를 찾는다
2. b11_boost_by_hand.py에서 학습률 = 0.3을 1.0으로 바꿔 실행한다. 5단계 뒤 오차는 어떻게 되나? 그 숫자만 보고 1.0이 낫다고 할 수 있을까?
3. b14_kmeans_k.py를 실행해 관성과 실루엣 점수로 k를 골라 본다

```
cd examples/vol2/03-classic-ml && uv run python b11_boost_by_hand.py
```

다음 중 맞는 설명은 무엇일까요?

Q

가 결정 트리는 깊게 키울수록 시험 정확도가 계속 오른다

나 부스팅은 앞 나무가 틀린 만큼을 다음 나무가 차례로 고친다

다 k-means의 0번 무리는 언제나 가장 큰 무리다

라 결정 트리도 k-means처럼 표준화가 꼭 필요하다

8차시 정리

결정 트리는 스무고개. 읽기 쉽지만 깊으면 외운다. max_depth, min_samples_leaf로 자른다

랜덤 포레스트는 서로 다른 나무들의 다수결, **그래디언트 부스팅**은 잔차를 조금씩 고쳐 가는 오답 노트

k-means는 깃발 옮기기 반복. k는 엘보와 실루엣으로, 거리를 쓰니 표준화가 필수

다음 차시: 99% 정확한 엉터리 검사기 · 모델을 제대로 채점하기

8차시 · 정리 · 복습: 책 3.4절 ~ 3.6절

9차시

09

평가 지표와 교차 검증

책 3.7절 ~ 3.9절

학습 목표

- 혼동 행렬에서 정밀도·재현율·F1을 직접 계산하고 문턱의 시소를 설명한다
- ROC-AUC와 MAE·RMSE·R^2를 상황에 맞게 고른다
- K-겹 교차 검증과 StratifiedKFold로 점수를 믿을 만하게 잰다

3.7 99% 정확한 엉터리 검사기

```
y = np.zeros(1000, dtype=int)
y[rng.choice(1000, size=10, replace=False)] = 1

게으른_검사기 = DummyClassifier(strategy="most_frequent").fit(X, y)
예측 = 게으른_검사기.predict(X)          # 누구에게나 "음성"
print("양성 판정 수:", 예측.sum())
print(f"정확도: {accuracy_score(y, 예측):.3f}")
print(f"감염자 중 찾아낸 비율(재현율): {recall_score(y, 예측):.3f}")
```

```
양성 판정 수: 0
정확도: 0.990
감염자 중 찾아낸 비율(재현율): 0.000
```

감염자 10명을 **한 명도** 못 찾았는데 정확도 0.990. 새 모델은 언제나 **Dummy 기준선**부터 잰다

3.7 혼동 행렬: 네 가지 칸

	예측: 음성(0)	예측: 양성(1)
실제: 음성(0)	TN 진짜 음성	FP 거짓 양성 (헛경보)
실제: 양성(1)	FN 거짓 음성 (놓침)	TP 진짜 양성

```
[[89  1]
 [ 4 49]]
TN=89, FP=1, FN=4, TP=49
직접 계산: 정밀도 0.980, 재현율 0.925, F1 0.951
sklearn : 정밀도 0.980, 재현율 0.925, F1 0.951
```

```
정밀도 = tp / (tp + fp)
재현율 = tp / (tp + fn)
f1 = 2 * 정밀도 * 재현율 / (정밀도 + 재현율)
```

정밀도: "양성" 판정을 믿어도 되나? 50번 중 49번

재현율: 환자를 빠뜨리지 않나? 53명 중 49명

F1: 둘의 조화 평균. 한쪽만 좋으면 크게 떨어진다

3.7 문턱을 움직이면: 정밀도와 재현율의 시소

```
for 문턱 in [0.9, 0.5, 0.2, 0.05]:
    예측 = (악성_확률 >= 문턱).astype(int)  # 문턱을 넘으면 양성 판정
    tn, fp, fn, tp = confusion_matrix(y_시험, 예측).ravel()
    print(f"{문턱:>4} {fp:>4} {fn:>4} {tp / (tp + fp):>8.3f} {tp / (tp + fn):>8.3f}")
```

```
문턱    FP    FN    정밀도    재현율
 0.9     0    10     1.000     0.811
 0.5     1     4     0.980     0.925
 0.2     3     2     0.944     0.962
0.05    11     0     0.828     1.000
```

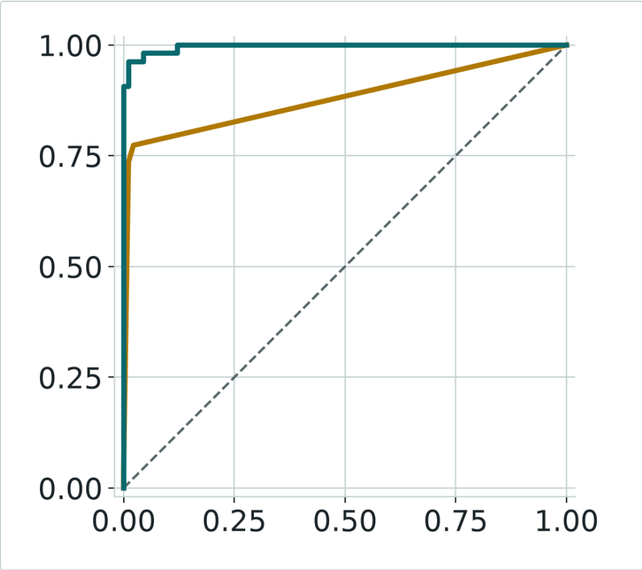
암 검진: 놓치면(FN) 치료 시기를 잃는다. 재현율 중시, 문턱을 낮춘다

스팸 필터: 헛경보(FP)가 더 큰 문제. 정밀도 중시

9차시 · 3.7 문턱을 움직이면 생기는 일 · 예제 03-classic-ml/b18_threshold.py (핵심 부분)

3.7 ROC-AUC: 문턱과 상관없이 매기는 점수

```
# AUC의 뜻: 악성 한 명과 악성 아닌 한 명을 뽑았을 때 악성 쪽 점수가 더 높을 확률
악성, 정상 = 점수[y_시험 == 1], 점수[y_시험 == 0]
이김 = (악성[:, None] > 정상[None, :]).mean() + 0.5 * (악성[:, None] == 정상[None, :]).mean()
```




```
아무 정보 없이 찍기: ROC-AUC 0.500
질문 두 번짜리 나무: ROC-AUC 0.880
로지스틱 회귀: ROC-AUC 0.996
짝 4770개를 직접 비교한 비율: 0.996
```

가로: 거짓 경보율(FPR), 세로: 재현율(TPR). **청록** 로지스틱 회귀, **노랑** 나무, 점선은 찍기(0.5). 거짓 경보율 0.011에서 재현율이 벌써 0.962

9차시 · 3.7 ROC-AUC · 예제 03-classic-ml/b19_roc_auc.py (핵심 부분, 곡선은 roc_curve 출력을 그림으로)

3.7 한눈에 보는 성적표

```
print(classification_report(y_시험, 모델.predict(X_시험),
                            target_names=["benign(0)", "malignant(1)"], digits=3))
```

```
              precision    recall  f1-score   support

   benign(0)      0.957     0.989     0.973        90
malignant(1)      0.980     0.925     0.951        53

    accuracy                          0.965       143
   macro avg      0.968     0.957     0.962       143
weighted avg      0.966     0.965     0.965       143
```

- malignant(1) 줄 = 앞에서 계산한 0.980, 0.925, 0.951
- **support**: 실제 개수
- 불균형일 때는 **macro avg**를 본다

3.8 회귀 모델 평가하기: MAE, RMSE, R²

```
오차 = y_시험 - 예측
mae = np.abs(오차).mean()
rmse = np.sqrt((오차 ** 2).mean())
r2 = 1 - (오차 ** 2).sum() / (
    (y_시험 - y_시험.mean()) ** 2).sum()
```

```
직접 계산: MAE 41.5, RMSE 53.4, R² 0.485
sklearn : MAE 41.5, RMSE 53.4, R² 0.485
평균으로 찍기: MAE 65.5, RMSE 74.9, R² -0.014
한 명을 300 틀리면: MAE 42.8, RMSE 56.8
```

지표	단위	큰 실수에	이럴 때
MAE	목표값과 같음	덜 민감	"평균 몇 만큼 틀린다"고 설명할 때
RMSE	목표값과 같음	민감	큰 실수를 특히 피해야 할 때, 모델끼리 비교할 때
R²	없음(1이 최고)	민감	데이터가 다른 문제끼리 "얼마나 설명하나"를 비교할 때

3.9 교차 검증: 모의고사는 한 번보다 다섯 번

```
시험 정확도: [0.916, 0.944, 0.93, 0.951, 0.937, 0.944, 0.951, 0.923, 0.916, 0.916]
가장 낮을 때 0.916, 가장 높을 때 0.951
```

```
접기 = KFold(n_splits=5, shuffle=True, random_state=0)
for 번호, (학습_칸, 검증_칸) in enumerate(접기.split(X), start=1):
    나무 = DecisionTreeClassifier(max_depth=3, random_state=0)
    # 네 묶음으로 배우고
    나무.fit(X[학습_칸], y[학습_칸])
    # 남은 한 묶음으로 채점
    점수 = 나무.score(X[검증_칸], y[검증_칸])
```

```
1번째: 학습 455개, 검증 114개 → 정확도 0.965
2번째: 학습 455개, 검증 114개 → 정확도 0.930
3번째: 학습 455개, 검증 114개 → 정확도 0.912
4번째: 학습 455개, 검증 114개 → 정확도 0.939
5번째: 학습 456개, 검증 113개 → 정확도 0.938
평균 0.937 ± 0.017
```

모든 샘플이 정확히 한 번씩 채점에 쓰인다 · 평균과 흩어진 정도를 함께 본다

9차시 · 3.9 교차 검증 · 예제 03-classic-ml/b22_split_luck.py(위 출력), 03-classic-ml/b23_kfold.py(아래 코드와 출력) (핵심 부분, 주석 위치만 바꿈)

3.9 함수 하나로: cross_val_score, cross_validate

```
모델 = make_pipeline(StandardScaler(), LogisticRegression())
접기 = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

점수 = cross_val_score(모델, X, y, cv=접기)                # 기본 채점: 정확도
print("정확도 5개:", 점수.round(3), f"→ 평균 {점수.mean():.3f}")

결과 = cross_validate(모델, X, y, cv=접기, return_train_score=True,
                      scoring=["accuracy", "recall", "roc_auc"])
```

```
정확도 5개: [0.956 0.974 0.982 1.    0.982] → 평균 0.979
accuracy: 학습 0.988, 검증 0.979 ± 0.014
  recall: 학습 0.976, 검증 0.958 ± 0.031
 roc_auc: 학습 0.998, 검증 0.995 ± 0.006
```

스케일러를 따로 먼저 하지 않고 **파이프라인 통째로** 넣었다. 회차마다 학습 묶음으로만 배운다

3.9 StratifiedKFold: 묶음마다 비율 맞추기

```
레이블 앞부분과 뒷부분: [0 0 0 0 0] [2 2 2 2 2]
KFold(섞지 않음)
  검증 묶음의 품종별 개수: [50  0  0]
  검증 묶음의 품종별 개수: [ 0 50  0]
  검증 묶음의 품종별 개수: [ 0  0 50]
  정확도: [0. 0. 0.]
StratifiedKFold
  검증 묶음의 품종별 개수: [17 17 16]
  검증 묶음의 품종별 개수: [17 16 17]
  검증 묶음의 품종별 개수: [16 17 17]
  정확도: [0.98 0.94 0.98]
```

```
for 이름, 접기 in [
        ("KFold(섞지 않음)", KFold(n_splits=3)),
        ("StratifiedKFold", StratifiedKFold(n_splits=3))]:
```

1단원만 공부시키고 3단원 시험을 본 꼴. 분류는 **섞은 StratifiedKFold**가 기본

직접 실행 10분

1. b18_threshold.py를 실행하고, 암 검진 담당자라면 네 문턱 중 무엇을 고를지 이유와 함께 적는다
2. b25_stratified.py에서 KFold(n_splits=3)을 KFold(n_splits=3, shuffle=True, random_state=0)으로 바꿔 실행한다. 검증 묶음과 정확도는 어떻게 바뀌나?
3. b24_cross_val.py를 실행해, 재현율이 정확도보다 더 흔들리는 이유를 말해 본다

```
cd examples/vol2/03-classic-ml && uv run python b25_stratified.py
```

악성 판정 문턱을 0.5에서 0.2로 낮추면 어떻게 될까요?

Q

가 정밀도가 오르고 재현율이 떨어진다

나 놓친 환자(FN)는 줄고 헛경보(FP)는 는다

다 ROC-AUC가 오른다

라 혼동 행렬은 그대로다

9차시 정리

분류 지표: 드문 양성 앞에서 정확도는 속인다. 혼동 행렬 네 칸에서 정밀도·재현율·F1, 문턱의 시소, 문턱과 상관없는 ROC-AUC

회귀 지표: MAE는 설명하기 쉽고, RMSE는 큰 실수에 민감하고, R^2는 평균으로 찍기 대비 얼마나 나은지

교차 검증: 모의고사는 다섯 번. 분류는 섞은 StratifiedKFold, 전처리는 파이프라인째로

다음 차시: 다이얼 돌리기, 저장하기, 그리고 캘리포니아 집값

9차시 · 정리 · 복습: 책 3.7절 ~ 3.9절

10차시

10

하이퍼파라미터 탐색, 저장, 종합 실습

책 3.10절 ~ 3.13절

학습 목표

- GridSearchCV와 RandomizedSearchCV로 설정을 고른다
- 파이프라인을 버전과 함께 joblib으로 저장하고 불러온다
- 기준선 → 비교 → 탐색 → 시험 한 번의 흐름을 끝까지 해 본다

3.10 손잡이 조합을 모두 돌려 보기: GridSearchCV

```
파이프 = make_pipeline(StandardScaler(), KNeighborsClassifier())
후보 = {
    "kneighborsclassifier__n_neighbors": [1, 3, 5, 9, 15, 25],   # 단계이름__설정이름
    "kneighborsclassifier__weights": ["uniform", "distance"],
}
탐색 = GridSearchCV(파이프, 후보, scoring="f1",
                    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=0))
탐색.fit(X_학습, y_학습)                    # 6 × 2 = 12조합 × 5묶음 = 60번 학습
```

```
그 조합의 교차 검증 F1: 0.948
 n_neighbors  weights  mean_f1  rank
           3  uniform    0.948     1
           3 distance    0.948     1
           9 distance    0.943     3
           9  uniform    0.940     4
```

3.10 조합이 너무 많을 때: RandomizedSearchCV

```
분포 = {
    "learning_rate": loguniform(0.01, 0.3),   # 0.01~0.3을 자릿수 기준으로 고르게
    "max_leaf_nodes": randint(4, 32),         # 4 이상 32 미만 정수
    "min_samples_leaf": randint(5, 40),
}
탐색 = RandomizedSearchCV(HistGradientBoostingClassifier(random_state=0), 분포,
                          n_iter=15, scoring="roc_auc", random_state=0,
                          cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=0))
탐색.fit(X_학습, y_학습)                      # 15조합 × 5묶음 = 75번 학습
```

```
learning_rate=0.2790, max_leaf_nodes=28, min_samples_leaf=34
교차 검증 ROC-AUC: 0.9891
시험 세트 ROC-AUC: 0.9960
```

후보를 목록이 아니라 **분포**로. 9차시 로지스틱 회귀의 ROC-AUC(0.996)와 비슷한 수준

10차시 · 3.10 RandomizedSearchCV · 예제 03-classic-ml/b27_random.py (핵심 부분, 실행 약 28초)

3.11 모델 저장하고 불러오기: joblib

```
폴더 = Path("models")
폴더.mkdir(exist_ok=True)
경로 = 폴더 / "cancer_lr.joblib"
joblib.dump({"model": 모델, "sklearn_version": sklearn.__version__}, 경로)

꾸러미 = joblib.load(경로)                    # 다른 프로그램에서 불러온다고 생각하자
불러온_모델 = 꾸러미["model"]
print("확률까지 모두 같은가:", (모델.predict_proba(X) == 불러온_모델.predict_proba(X)).all())
```

```
저장: models/cancer_lr.joblib (2,342바이트)
저장할 때 버전: 1.9.1 / 지금 버전: 1.9.1
예측 비교(10명): [1 1 0 1 0 0 0 0 0 0] [1 1 0 1 0 0 0 0 0 0]
확률까지 모두 같은가: True
단계 이름: ['standardscaler', 'logisticregression']
```

파이프라인 통째로, 버전과 함께. 인터넷에서 받은 모르는 파일은 절대 load하지 않는다

3.12 언제 딥러닝 대신 고전 머신러닝을 쓸까?

따져 볼 것	고전 머신러닝이 유리	딥러닝이 유리
데이터 모양	표(행 = 샘플, 열 = 특성)	사진, 소리, 글, 영상
데이터 양	수백~수십만 줄	수십만~수억 개
특성	사람이 의미를 아는 값(나이, 금액 등)	픽셀, 글자처럼 하나하나는 의미가 약한 값
설명 요구	결정 이유를 설명해야 함(대출 심사, 의료)	성능이 가장 중요함
자원	CPU 한 대, 몇 초~몇 분	GPU, 몇 시간~몇 주

실무의 순서: **기준선(Dummy) → 간단한 모델(선형·로지스틱) → 트리 앙상블(숲, 부스팅)** → 그래도 부족하고 데이터가 충분하면 딥러닝

3.13 실습 1~2단계: 데이터 받고 후보 비교하기

```
# 처음 한 번은 인터넷에서 받아 data/ 폴더에 저장하고, 다음부터는 거기서 읽는다
집 = fetch_california_housing(data_home=os.environ.get("BOOK_DATA_DIR", "data"), as_frame=True)
X, y = 집.data, 집.target                    # y: 구역의 집값 중앙값(단위 10만 달러)
```

```
평균 찍기: RMSE 1.156 ± 0.024, R² -0.000
선형 회귀: RMSE 0.720 ± 0.020, R² 0.612
최근접 이웃: RMSE 0.641 ± 0.017, R² 0.692
랜덤 포레스트: RMSE 0.511 ± 0.013, R² 0.805
부스팅: RMSE 0.471 ± 0.010, R² 0.834
```

1단계: 20,640구역 × 특성 8개. 학습 16,512, 시험 4,128은 **봉인**

2단계: 학습 세트에서만 5겹 교차 검증. 흔들림보다 모델 간 차이가 훨씬 크다

10차시 · 3.13 실습 · 예제 03-classic-ml/b29_practice_data.py(위, 약 3초), 03-classic-ml/b30_practice_compare.py(왼쪽 출력, 약 8초)

3.13 실습 3~4단계: 탐색하고, 시험은 딱 한 번

3단계 · b31 (8조합 × 5겹)

```
l2_regularization: 0.0027
learning_rate: 0.0893
max_iter: 470
max_leaf_nodes: 43
min_samples_leaf: 44
교차 검증 RMSE: 0.457  (기본 설정은 0.471)
```

4단계 · b32 (봉인 해제)

```
시험 RMSE 0.450, MAE 0.296, R² 0.845
예상  5.40만 달러 / 실제  4.77만 달러
예상 10.59만 달러 / 실제  4.58만 달러
예상 50.79만 달러 / 실제 50.00만 달러
```

```
최종 = HistGradientBoostingRegressor(
    learning_rate=0.0893, max_iter=470, max_leaf_nodes=43,
    min_samples_leaf=44, l2_regularization=0.0027, random_state=0)
최종.fit(X_학습, y_학습)                    # 학습 세트 전체로 학습
예측 = 최종.predict(X_시험)                 # 봉인했던 시험 세트, 딱 한 번
```

성능의 대부분은 **어떤 모델을 고르느냐**(0.720 → 0.471)에서, 탐색은 조금 더 짜내기(0.471 → 0.457)

10차시 · 3.13 실습 · 예제 03-classic-ml/b31_practice_search.py(실행 약 98초), 03-classic-ml/b32_practice_final.py(약 4초) (핵심 부분)

직접 실행 10분

1. b31_practice_search.py를 **먼저** 시작해 둔다(약 98초). 기다리는 동안 다른 창에서 b30(약 8초)과 b32(약 4초)를 실행한다
2. b26_grid.py에서 scoring="f1"을 scoring="recall"로 바꿔 실행한다. 가장 좋은 이웃 수는 어떻게 바뀌나?
3. b31이 끝나면 교차 검증 RMSE를 b32의 시험 RMSE와 견주어 본다

```
cd examples/vol2/03-classic-ml && uv run python b31_practice_search.py
```

시험 점수가 아쉬워 학습률을 바꿔 같은 시험 세트로 다시 재면?

Q

가 실력을 더 정확히 알게 된다

나 시험 세트가 답을 맞춰 가며 고친 문제가 되어 점수가 부푼다

다 교차 검증을 한 것과 같아진다

라 학습 세트로 다시 학습했다면 문제없다

10차시 정리

탐색: 다이얼이 적으면 GridSearchCV, 많으면 RandomizedSearchCV. 둘 다 학습 세트 안의 교차 검증으로

저장: 파이프라인 통째로 joblib.dump, 버전을 함께 기록, 모르는 파일은 열지 않기

흐름: 기준선 → 후보 비교 → 탐색 → 시험 세트는 마지막에 한 번. 성능은 대부분 모델 고르기에서 나온다

다음: 이틀 과정 마무리

이틀 동안 걸어온 길

차시	한 줄
1	규칙을 짜는 대신 데이터와 정답을 주고, 규칙은 프로그램이 배운다
2	가까운 이웃의 다수결로 첫 모델을 직접 만들었다
3	외우는 것과 일반화하는 것은 다르다. 처음 보는 데이터로 잰다
4	먼저 살펴보고, 나누고, 시험 문제가 새는 길을 막는다
5	빈칸 채우기, 글자를 숫자로, 단위 맞추기는 학습 데이터로 배운다
6	손질과 모델을 Pipeline 하나로. 드문 쪽과 시간 순서를 지켜 나눈다
7	모든 모델은 fit·predict·score. 직선의 계수를 읽고, 점수를 확률로 바꾼다
8	스무고개, 다수결, 오답 노트, 깃발 세 개
9	정확도는 속인다. 혼동 행렬과 문턱, 모의고사는 다섯 번
10	설정은 교차 검증으로 고르고, 저장하고, 시험 세트는 마지막에 한 번

다음 과정: 2권 4장 텐서부터 PyTorch로